UNIVERSIDAD AMERICANA (UAM)

Programación de Aplicaciones de Escritorio

LABORATORIO GUIADO
CONCURRENCIA EN JAVA Y JAVAFX

Informe de evidencias, pruebas y reflexión final

Integrante	Responsabilidad evidenciada en el repositorio
Andy Diaz	Thread/Runnable, aplicación base e integración inicial.
Anthony González	Monitoreo e interpretación de estados de hilos.
Luis Aburto	Sincronización, AtomicInteger y pruebas de concurrencia.
Mia Flores	Controladores, FXML y control de la integración JavaFX.

Fecha: 25 de octubre de 2026

1. Propósito y preparación del entorno

El laboratorio tiene como propósito construir una aplicación JavaFX capaz de ejecutar tareas en segundo plano, observar estados de hilos, proteger un recurso compartido y permitir la cancelación controlada sin bloquear la interfaz gráfica. El proyecto entregado utiliza Java 21, JavaFX 21.0.6 y Maven, superando el requisito mínimo de JDK 17.

Estructura principal verificada en el proyecto:

- app: clase principal de la aplicación JavaFX.
- controller: lógica de interacción con la interfaz.
- tareas: Thread, Runnable, monitoreo y pruebas de concurrencia.
- modelo: contador compartido y mecanismos de sincronización.

Flujo general de ejecución

Iniciar → crear tareas/hilos → trabajo en segundo plano → actualización segura de JavaFX → Detener/Finalizar → Reiniciar

Las operaciones largas se realizan fuera del JavaFX Application Thread. Las actualizaciones visuales se envían al hilo gráfico mediante Platform.runLater(), evitando que la ventana quede congelada durante el procesamiento.

2. Estación 1. Thread y Runnable

La clase ProcesoVentanilla implementa Runnable y es utilizada por dos objetos Thread independientes. La prueba compara start() con una llamada directa a run(). Con start() los dos hilos pueden avanzar de forma concurrente; con run() el método se ejecuta secuencialmente en el hilo que lo invoca.

Resultados reproducidos en tres ejecuciones

Ejecución	Primeros eventos con start()	Tiempo start()	Tiempo run()
1	A1 → B1 → A2 → B2 …	2523 ms	5004 ms
2	B1 → A1 → A2 → B2 …	2519 ms	5003 ms
3	A1 → B1 → A2 → B2 …	2518 ms	5004 ms

El orden cambia entre ejecuciones porque el planificador del sistema operativo decide cuándo obtiene tiempo de CPU cada hilo. Por ello, no debe asumirse un orden fijo. start() tampoco significa que el trabajo terminó antes de devolver el control: únicamente solicita el inicio del nuevo hilo y retorna mientras la ejecución continúa en segundo plano.

3. Evidencias de ejecución

Evidencia 1. Dos ventanillas concurrentes e interfaz responsiva

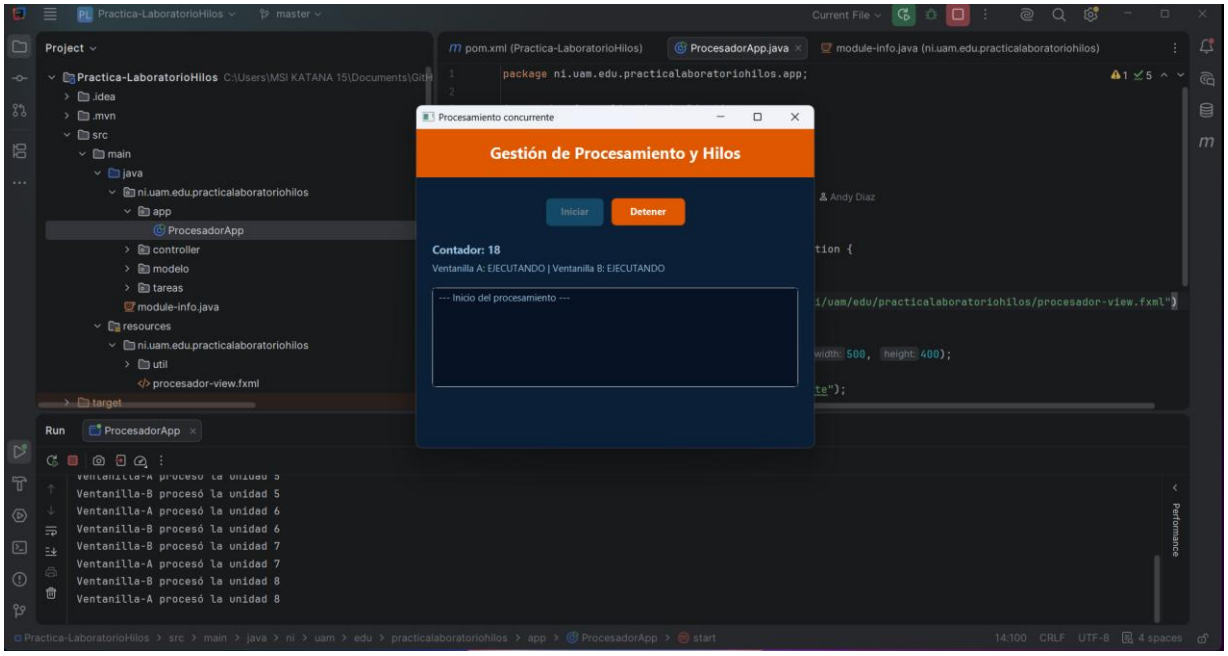


Figura 1. Aplicación JavaFX durante una ejecución concurrente de las dos ventanillas.

Se observa el botón Iniciar deshabilitado durante la sesión, el botón Detener disponible, el contador en aumento y ambas ventanillas en estado EJECUTANDO. La consola también muestra unidades procesadas por Ventanilla-A y Ventanilla-B, evidenciando trabajo concurrente sin bloquear la interfaz.

Evidencia 2. Estados del hilo

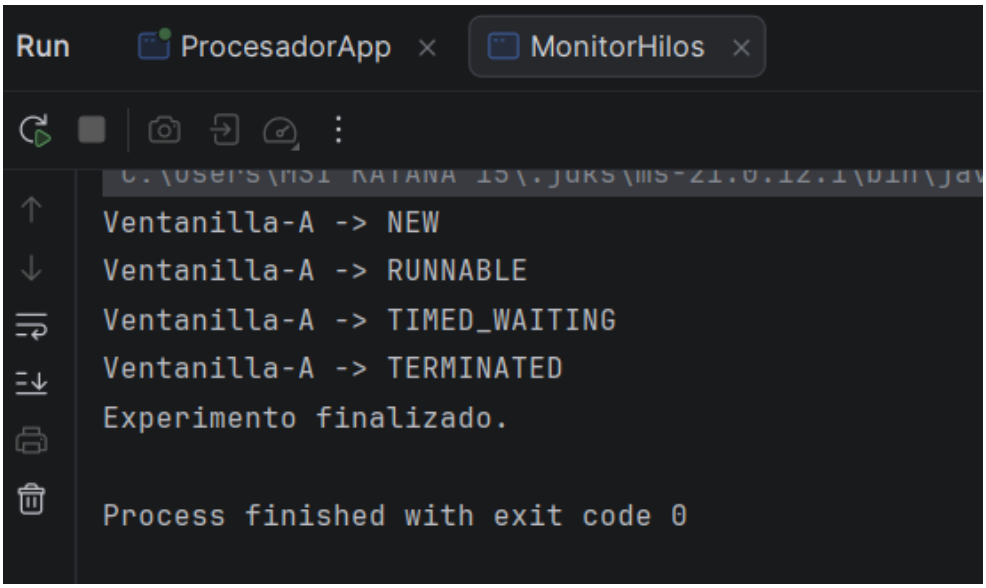


Figura 2. Monitoreo de estados NEW, RUNNABLE, TIMED_WAITING y TERMINATED.

El monitor registra el ciclo de vida del hilo. NEW corresponde al hilo creado pero no iniciado; RUNNABLE indica que está disponible para ejecutarse; TIMED_WAITING aparece durante una espera con tiempo, como Thread.sleep(); y TERMINATED confirma que el método run() finalizó.

La consulta getState() representa únicamente una instantánea: el estado puede cambiar inmediatamente después de ser leído debido a la planificación de hilos. Por esa razón no debe interpretarse como una garantía del estado futuro.

Evidencia 3. Condición de carrera y sincronización

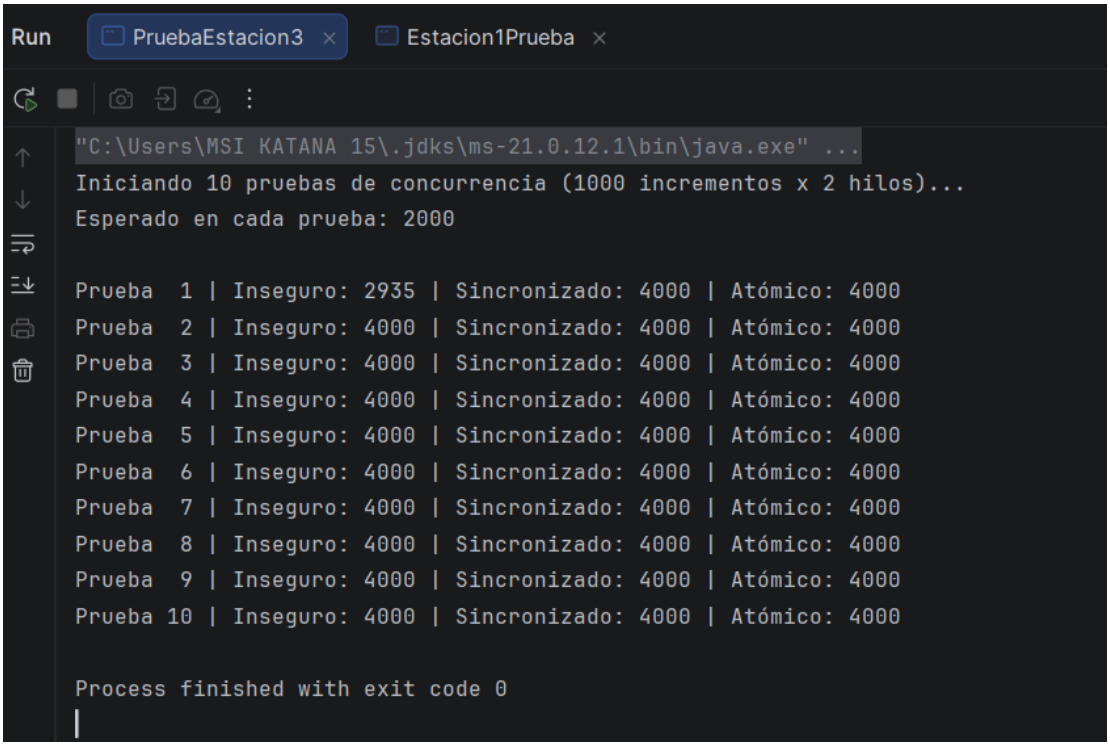


Figura 3. Prueba con carga aumentada para evidenciar la condición de carrera.

Para hacer visible la condición de carrera se aumentó la carga a 2000 incrementos por cada hilo. En la primera prueba el contador inseguro obtuvo 2935, mientras que el contador sincronizado y el AtomicInteger alcanzaron correctamente 4000. Esto demuestra que total++ no es una operación atómica y puede perder actualizaciones cuando varios hilos modifican el mismo valor al mismo tiempo.

Nota: la línea superior de la consola conserva el mensaje original “Esperado en cada prueba: 2000”. Al ejecutar 2000 incrementos por cada uno de los dos hilos, el valor esperado real de esta evidencia es 4000, coincidente con los resultados sincronizado y atómico.

Evidencia 4. Cancelación cooperativa

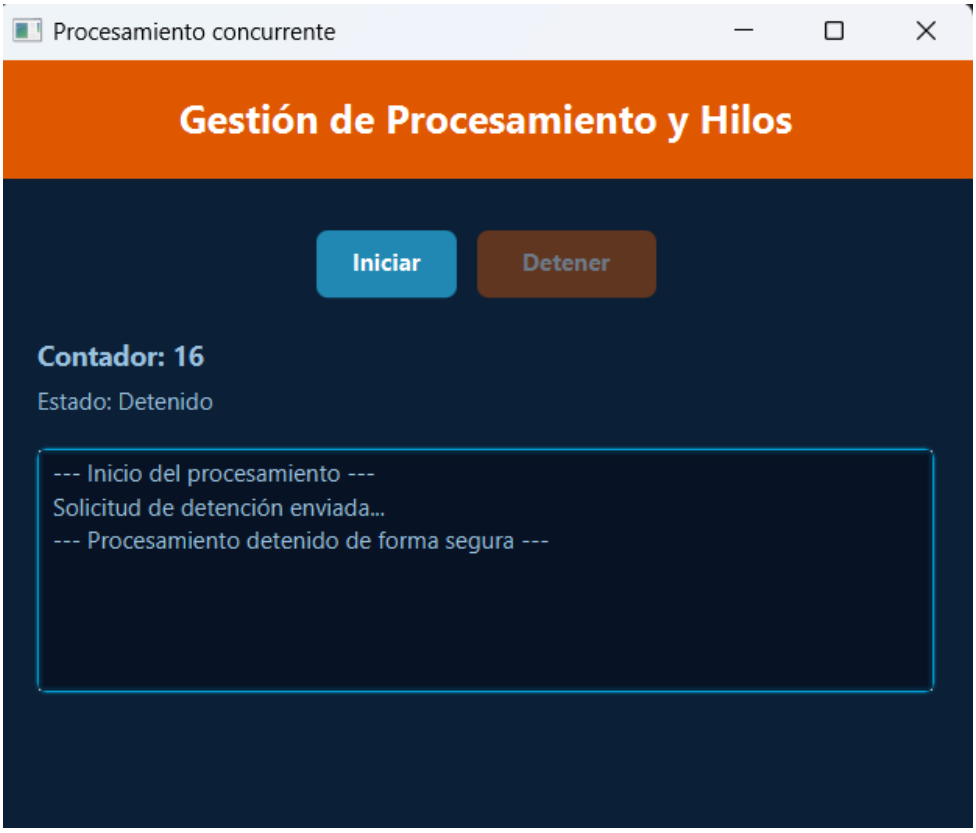


Figura 4. Detención controlada solicitada desde la interfaz JavaFX.

La interfaz registra la solicitud de detención y posteriormente confirma “Procesamiento detenido de forma segura”. La solución utiliza una bandera volatile e interrupt(), sin recurrir a Thread.stop(), y permite que cada hilo abandone su trabajo de manera controlada.

Evidencia 5. Reinicio de una nueva sesión



Figura 5. Nueva sesión iniciada después de una detención segura.

El registro visual muestra una detención completada y, después, un nuevo “Inicio del procesamiento”. La nueva ejecución vuelve a colocar ambas ventanillas en estado EJECUTANDO, lo que evidencia que no se inicia una sesión nueva hasta haber completado la detención de los hilos anteriores.

4. Tabla de pruebas y hallazgos

Prueba	Resultado esperado	Resultado observado	Estado
Dos ventanillas	Ambas trabajan y la ventana responde.	Las dos ventanillas aparecen EJECUTANDO; el contador avanza y la UI permanece operativa.	Cumple
Conteo concurrente	Total correcto sin pérdidas.	El contador inseguro falló bajo carga (2935); synchronized y AtomicInteger alcanzaron 4000.	Cumple
Cancelación	Las tareas finalizan cooperativamente.	Se registra solicitud de detención y finalización segura sin Thread.stop().	Cumple
Reinicio	No quedan hilos de la sesión anterior.	Después de detener, la aplicación permite iniciar una nueva sesión.	Cumple
Estados	Observaciones explicadas con evidencia.	Se observaron NEW, RUNNABLE, TIMED_WAITING y TERMINATED.	Cumple

5. Análisis técnico por estación

5.1 ¿Por qué cambia el orden de los mensajes?

Los hilos se ejecutan de manera concurrente y el sistema operativo decide cuándo ejecuta cada uno. Aunque ambos realizan el mismo número de iteraciones y utilizan tiempos similares, no existe garantía de

que Ventanilla-A siempre se ejecute antes que Ventanilla-B. Esto explica la variación observada entre ejecuciones.

5.2 Estados de hilo

Estado	Significado
NEW	El objeto Thread fue creado pero todavía no se ha llamado start().
RUNNABLE	El hilo está listo o ejecutándose según la planificación del sistema.
BLOCKED	Espera adquirir un monitor/bloqueo que otro hilo mantiene.
WAITING	Espera indefinidamente una acción de otro hilo.
TIMED_WAITING	Espera durante un tiempo determinado, por ejemplo con sleep().
TERMINATED	El método run() terminó y el hilo ya no ejecutará más trabajo.

5.3 ¿Qué cambió con synchronized y AtomicInteger?

Sin protección, dos hilos pueden leer el mismo valor, incrementarlo y escribir resultados que se sobrescriben. synchronized hace que la sección crítica sea ejecutada por un hilo a la vez. AtomicInteger ofrece una alternativa atómica para operaciones sencillas de incremento. Ambos mecanismos evitan la pérdida de actualizaciones observada en el contador inseguro.

synchronized no reemplaza Platform.runLater(): el primero protege datos compartidos entre hilos; el segundo garantiza que las modificaciones de controles JavaFX se ejecuten en el JavaFX Application Thread.

5.4 Interrupción y finalización

Solicitar interrupt() no garantiza que el hilo ya haya terminado. La interrupción es una señal cooperativa y el hilo debe comprobarla o reaccionar a InterruptedException. Al capturar InterruptedException se restablece el indicador mediante Thread.currentThread().interrupt() y se abandona la ejecución de forma ordenada; ignorarla podría provocar que el trabajo continúe cuando el usuario ya solicitó detenerlo.

5.5 Escalabilidad de las actualizaciones visuales

Para una carga pequeña es aceptable utilizar Platform.runLater() por actualización. Con 100 000 solicitudes, miles de eventos podrían acumularse en la cola gráfica. Una mejora sería agrupar actualizaciones: los hilos solo modifican un contador seguro y la interfaz lee el valor de forma periódica mediante Timeline, AnimationTimer o una actualización por lotes. De esta forma se reduce el número de cambios visuales sin perder el total procesado.

6. Retos de análisis colaborativo

¿Qué riesgos surgen si dos integrantes modifican el mismo controlador JavaFX al mismo tiempo?

Pueden aparecer conflictos de Git, pérdida accidental de cambios o integración de lógica incompatible. La estrategia recomendada es dividir responsabilidades, trabajar en ramas individuales, mantener cambios pequeños y realizar la integración mediante Pull Request revisado por el líder.

¿Por qué un Pull Request que compila puede seguir siendo incorrecto desde el punto de vista de concurrencia?

Compilar solo verifica sintaxis y tipos. Una condición de carrera, un bloqueo de la interfaz, una cancelación incompleta o un deadlock pueden aparecer únicamente durante la ejecución. Por ello el Pull Request debe incluir pruebas reales de comportamiento concurrente.

¿Qué evidencias demuestran una detención controlada y una interfaz no bloqueada?

El estado visible cambia a detención/detenido, los hilos responden a interrupt(), no se utiliza Thread.stop(), el botón de la interfaz sigue respondiendo y posteriormente puede iniciarse una nueva sesión sin actividad de los hilos anteriores.

¿Cómo resolver un conflicto de Git sin sobrescribir el trabajo de otro integrante?

Primero se actualiza la rama con los cambios remotos, se revisan los bloques marcados como conflicto y se combinan de forma consciente las partes necesarias de ambas versiones. Después se compila y se ejecutan pruebas antes de confirmar la resolución con un commit. No se debe elegir una versión completa sin revisar qué funcionalidad se perdería.

7. Reflexión final

1. ¿Qué error apareció antes de sincronizar? El problema principal fue una condición de carrera sobre el contador compartido. El incremento total++ se compone de varias operaciones y dos hilos pueden interferir entre sí. Al aumentar la prueba a 2000 incrementos por hilo se obtuvo 2935 en el contador inseguro, mientras el resultado correcto era 4000. Esto demostró una pérdida real de actualizaciones.

2. ¿Cómo se evitó bloquear JavaFX? El procesamiento se trasladó a hilos de trabajo separados. La interfaz no ejecuta sleep() ni join() dentro de los eventos de JavaFX. Las modificaciones visuales se realizan mediante Platform.runLater(), permitiendo que el JavaFX Application Thread continúe atendiendo botones, repintado y eventos del usuario.

3. ¿Qué condiciones se verificaron para detener y reiniciar? La detención se solicita de forma cooperativa mediante una bandera volatile e interrupt(). Cada tarea comprueba el estado de interrupción y maneja InterruptedException sin ignorarla. La nueva sesión se habilita después de que la sesión anterior ha terminado su proceso de detención, evitando mezclar hilos de ejecuciones diferentes.

4. ¿Qué se mejoraría para procesar 100 000 solicitudes? Se reduciría la frecuencia de actualizaciones visuales. En lugar de generar un Platform.runLater() por unidad procesada, los hilos actualizarían un contador seguro y la interfaz refrescaría el valor de forma periódica o por lotes. También se mantendrían mecanismos atómicos o sincronizados para los recursos compartidos y se mediría el rendimiento con cargas mayores.

8. Evidencia de trabajo colaborativo en GitHub

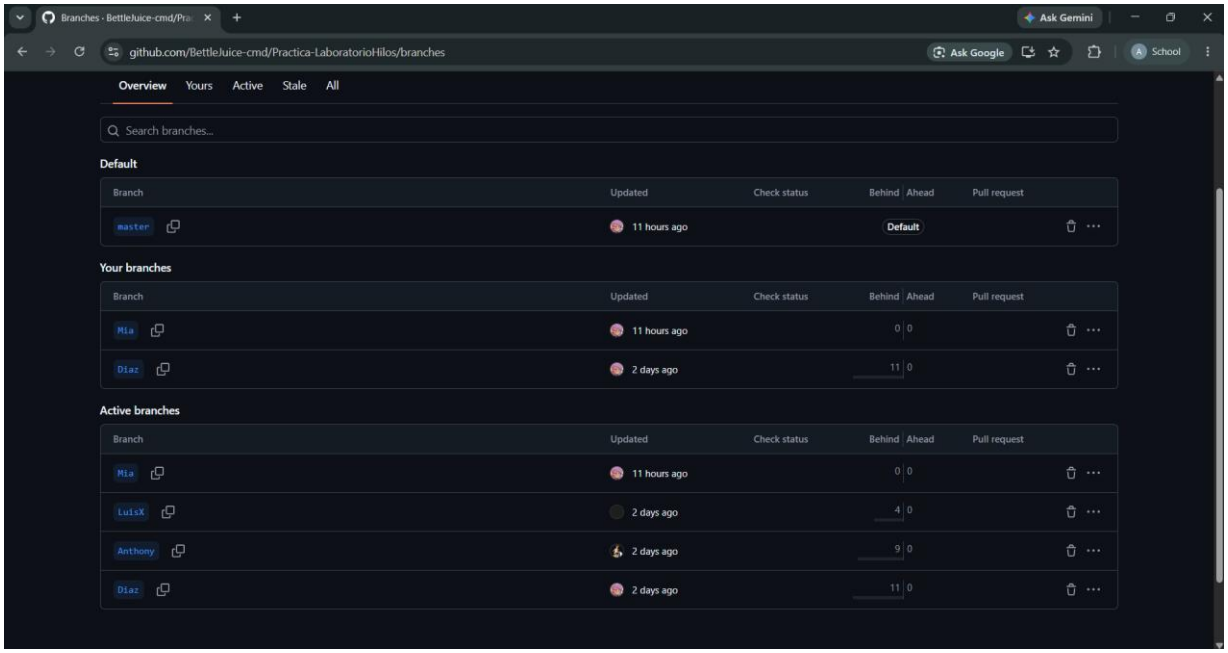
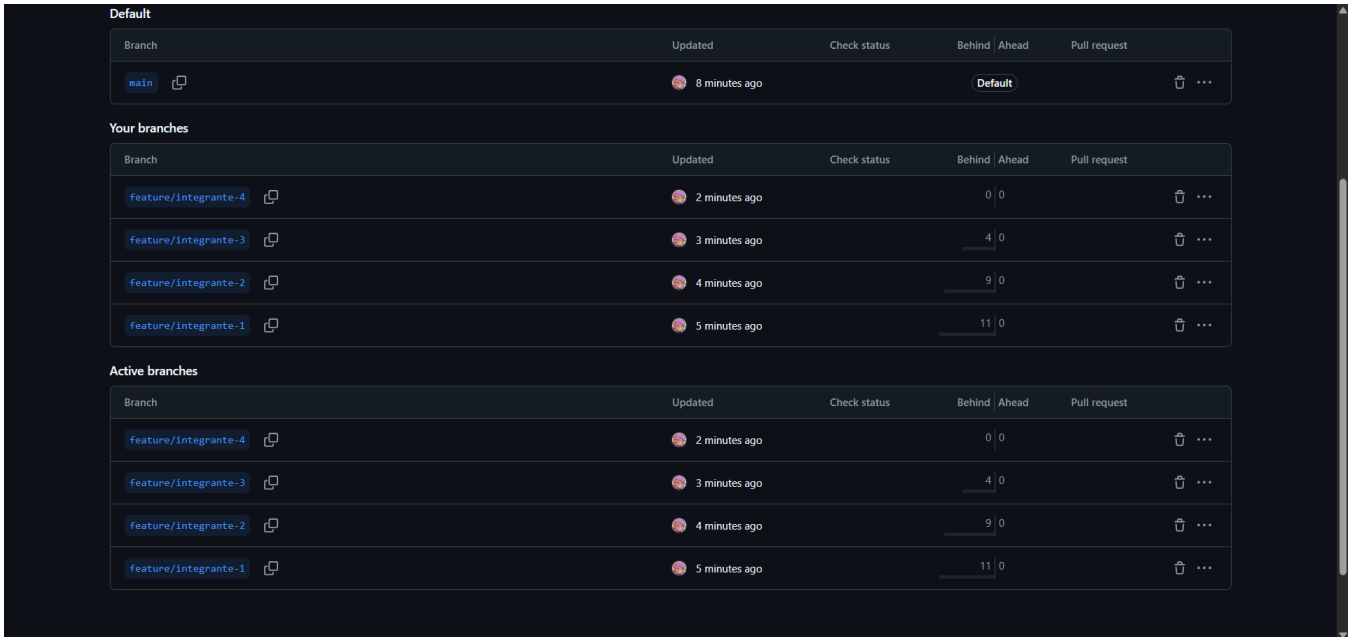


Figura 6. Ramas del repositorio y participación individual del equipo.



La evidencia muestra una rama estable y ramas individuales de los cuatro integrantes. El historial del proyecto entregado contiene commits identificados de Andy Diaz, Anthony González, Luis Aburto y Mia Flores, asociados a Thread/Runnable, monitoreo de estados, sincronización y controladores/FXML respectivamente. Esto permite relacionar las contribuciones con funcionalidades reales del laboratorio.

Distribución de responsabilidades evidenciada

Integrante	Área principal	Evidencia en commits
Andy Diaz	Thread/Runnable e integración inicial	Implementación de ProcesoVentanilla, Estacion1Prueba, ProcesadorApp y controlador inicial.
Anthony González	Estados de hilos	Implementación de MonitorHilos e integración de cambios.
Luis Aburto	Sincronización	ContadorCompartido, AtomicInteger y PruebaEstacion3.
Mia Flores	JavaFX/FXML	Controladores, procesamiento y vista FXML.

9. Conclusión

El laboratorio permitió comprobar que la concurrencia requiere controlar tanto el acceso a datos compartidos como la interacción con la interfaz gráfica. Las pruebas demostraron una condición de carrera real en el contador sin protección y confirmaron que synchronized y AtomicInteger mantienen la consistencia. También se verificaron estados del ciclo de vida de los hilos, cancelación cooperativa y reinicio de sesiones sin bloquear JavaFX. En conjunto, la solución muestra el uso práctico de Thread, Runnable, interrupciones, sincronización y actualización segura de controles JavaFX.

Documento elaborado a partir del proyecto entregado, las capturas de ejecución proporcionadas y las pruebas reproducibles del código del laboratorio.